AGENTIC AI SYSTEMS · HARNESS, LECTURE 2

# Advanced Harness Architectures and Their Evaluation

Yonathan Efroni

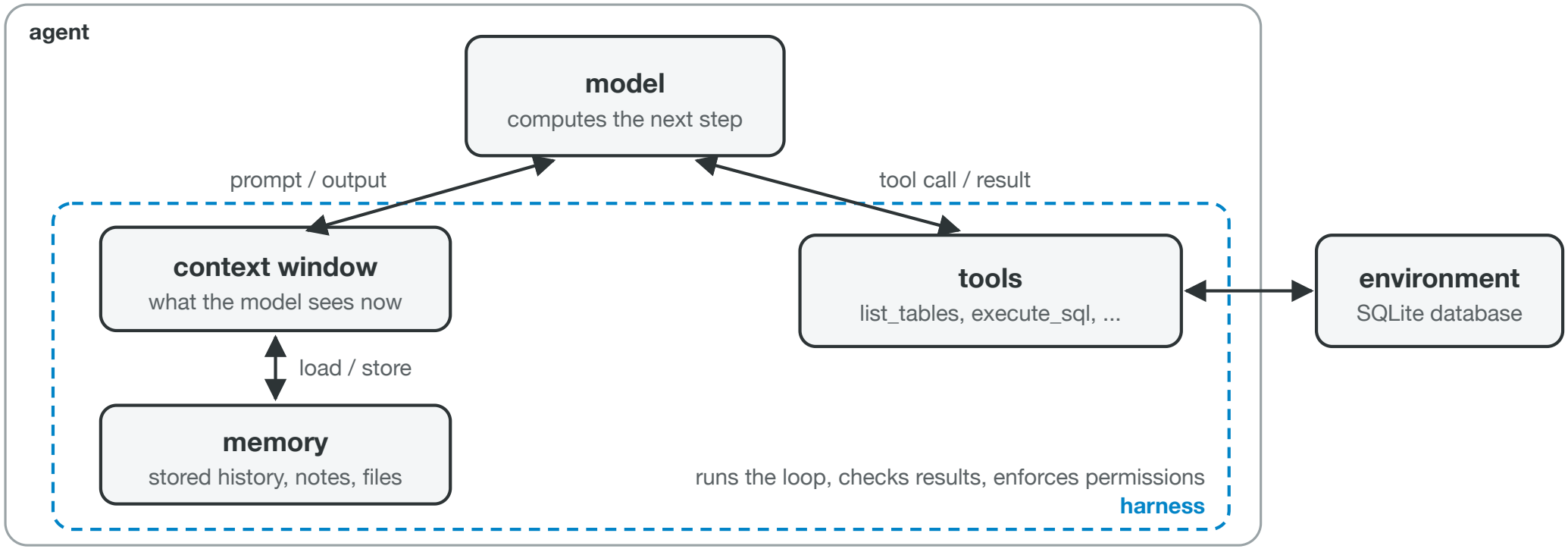
Tel Aviv University

RECAP · HARNESS, LECTURE 1

# Foundations of agent harnesses

# An agent is a model plus a harness

The model computes; the harness manages its context, memory and tools.




*Our SQL agent: 95% of tasks solved vs 60% single-shot, at about 4× the cost.*

# From Lecture 1 to today

Today we dive deeper and test additional design choices of the harness.

| Lecture 1 | SQL agent | Today |
|---|---|---|
| **Execution loop** | one short loop | H1 AutoResearch, H2 plan-and-execute, H3 context-managed |
| **Tools** | `execute_sql`, and more | `evaluate_equation`, delegation |
| **Context** | full history | H3 curated context, fresh executors |
| **Memory** | same as context | H3 persistent record |
| **Verification** | execution check, LLM judge | training fit, hidden tests, and more |
| **Evaluation** | success, cost, and more | ID and OOD fit, and more |

PART 1

# From components to architectures

# Advanced design choices

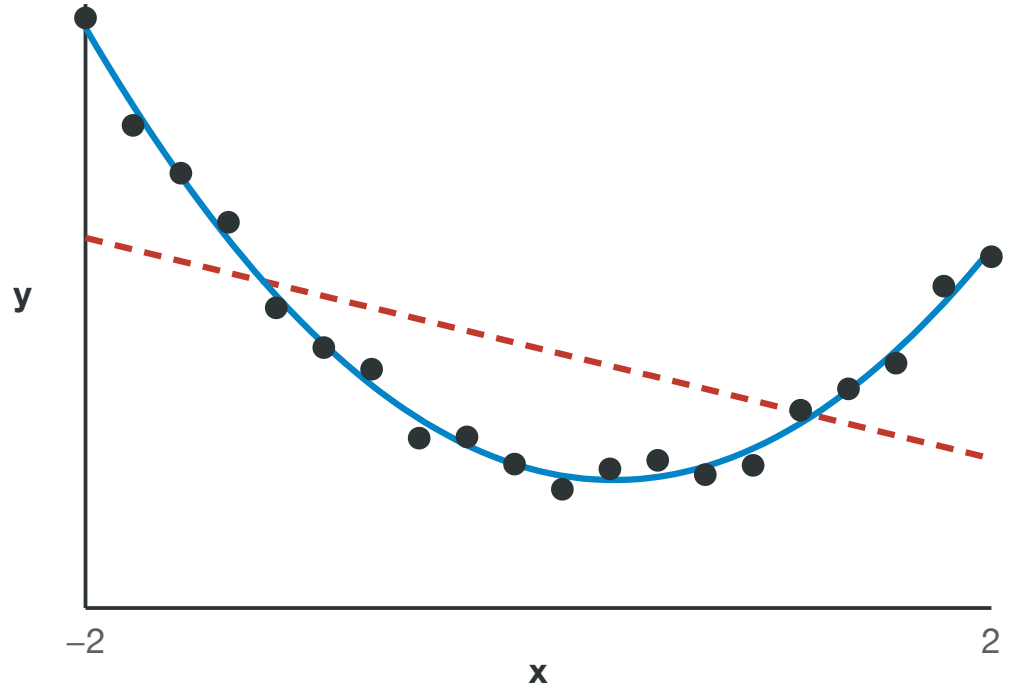
Organizing the components is still an open design decision.

- **Planning:** choose the next action reactively, or plan several investigations?
- **Multi-agent orchestration:** one agent, or several coordinated agents?
- **Context:** keep the full history, or retain only what is relevant?

When does added complexity improve performance enough to justify its cost?

# Running example: symbolic regression

Given data $D = \{(x_i, y_i)\}_{i=1}^{n}$, find an expression $f$ such that $y \approx f(x)$.




**$f_1(x) = ax + b$**

$R^2 = 0.27 \rightarrow$ try another form

**$f_2(x) = ax^2 + bx + c$**

$R^2 = 0.99$

$R^2$ returned by evaluate_equation

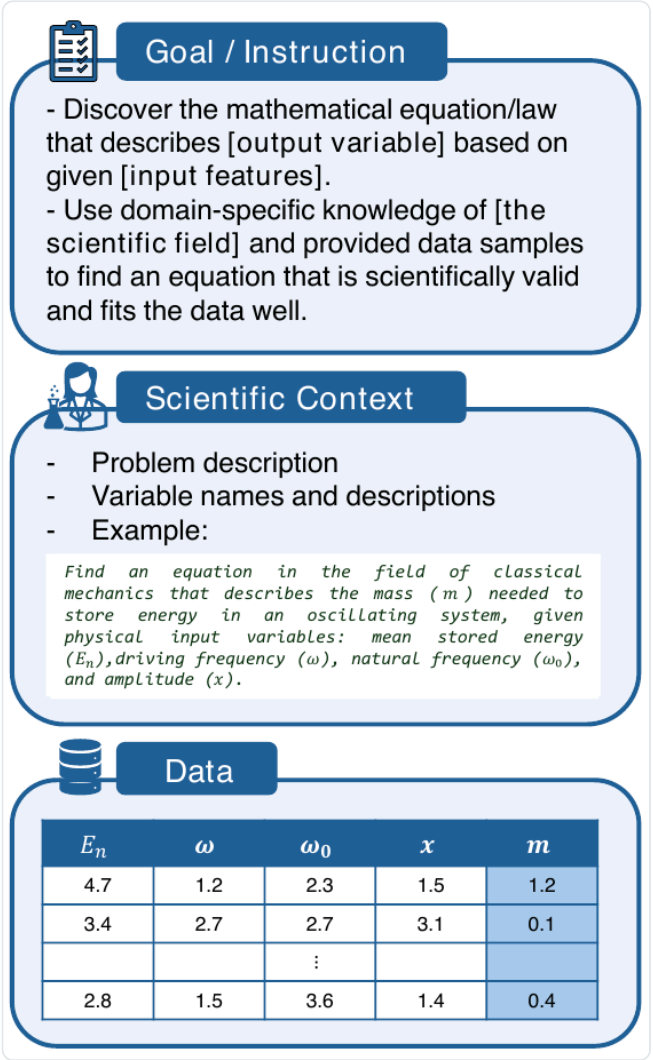
Shojaee et al., *LLM-SR: Scientific Equation Discovery via Programming with Large Language Models*, ICLR 2025
Shojaee et al., *LLM-SRBench: A New Benchmark for Scientific Equation Discovery with Large Language Models*, ICML 2025

# What the agent has

A task description, training data and one tool.

- **Task** $q$: goal, problem description, variable names
- **Training data** $D_{\text{train}} = \{(x_i, y_i)\}_{i=1}^{n}$
- **Tool:** `evaluate_equation` fits the free parameters $\theta$ of a candidate $f(x;\theta)$ and returns its training fit ($R^2$, NMSE) and the fitted constants

$$\hat{\theta} = \arg\min_{\theta} \sum_{i=1}^{n} \left(y_i - f(x_i;\theta)\right)^2$$



# The task as an MDP

| MDP | Symbolic regression |
|---|---|
| state $s_t$ | history $h_t$: the task and all observations so far |
| action $a_t$ | a candidate equation $f_t$, sent to `evaluate_equation` |
| transition | the tool returns $o_t$: training fit and fitted constants; $h_{t+1} = h_t \cup \{a_t, o_t\}$ |
| proxy reward | $R_{\text{ID}}(\hat{f})$: the fit in the training range, what the agent optimizes |
| true reward | $R_{\text{OOD}}(\hat{f})$: the fit beyond the training range, what we want |
| policy $\pi$ | $\pi_{M,H}$: model $M$ inside harness $H$ |

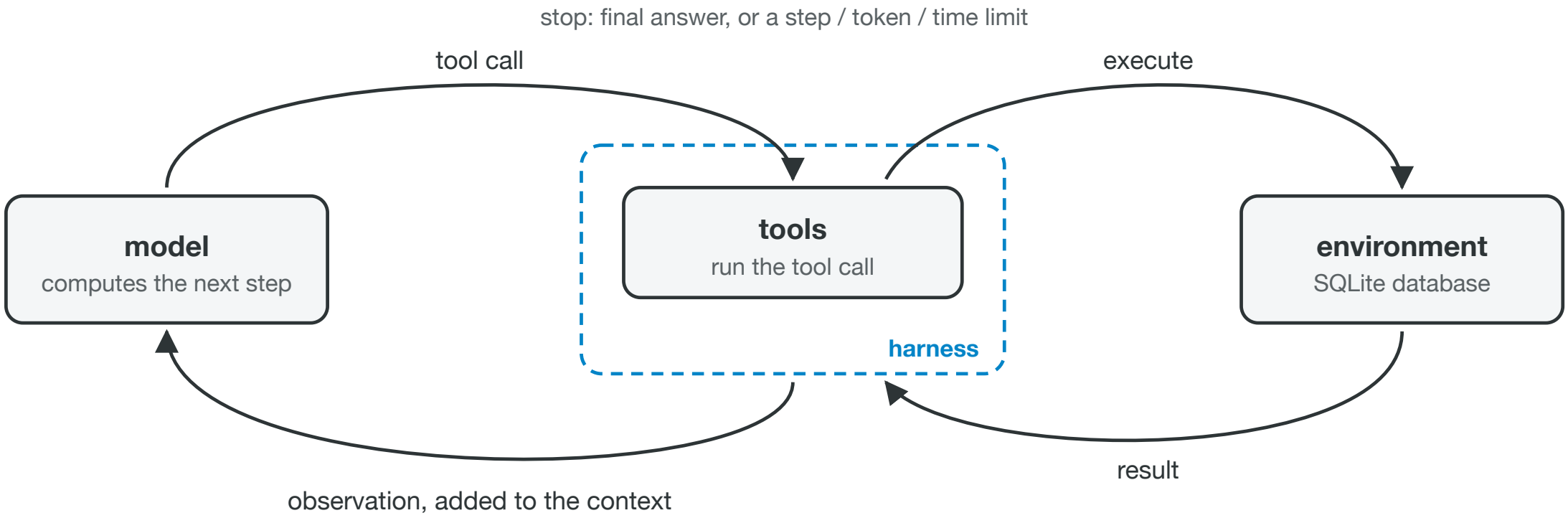
$$J(\pi_{M,H}) = \mathbb{E}\left[R_{\text{ID}}(\hat{f})\right], \qquad J^{\star}(\pi_{M,H}) = \mathbb{E}\left[R_{\text{OOD}}(\hat{f})\right]$$

PART 2

# Three harness architectures

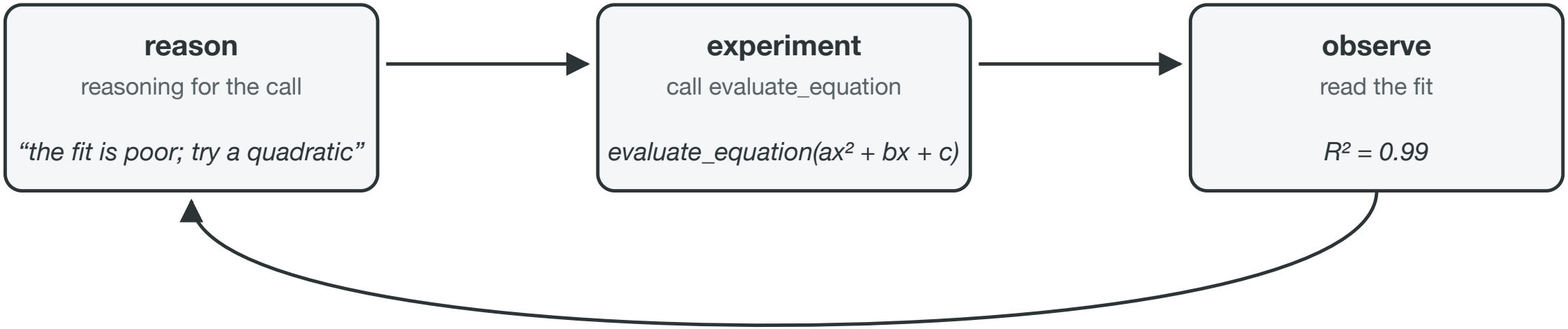
# Recall: the execution loop

In Lecture 1, the model proposed actions and the harness executed them.



# AutoResearch: the loop, applied to research

AutoResearch runs the same loop, with experiments as actions.




We test AutoResearch both with and without the reasoning step.

Think, then act: the ReAct pattern. Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, ICLR 2023

# AutoResearch: the algorithm

One agent picks each next experiment from the full history of experiments.

$$\text{reasoning}_t \sim M(\cdot \mid q, h_t), \qquad f_t \sim M(\cdot \mid q, h_t, \text{reasoning}_t),$$

$$o_t = \texttt{evaluate_equation}(f_t), \quad h_{t+1} = h_t \cup \{\text{reasoning}_t, f_t, o_t\}$$

```
h = [task]
while budget left and not finished:
    reasoning = M(h)              # 1. ask for reasoning
    f = M(h + [reasoning])        # 2. given it, choose the evaluation
    o = evaluate_equation(f)      # fit on training data
    h.append((reasoning, f, o))
return best f in h
```

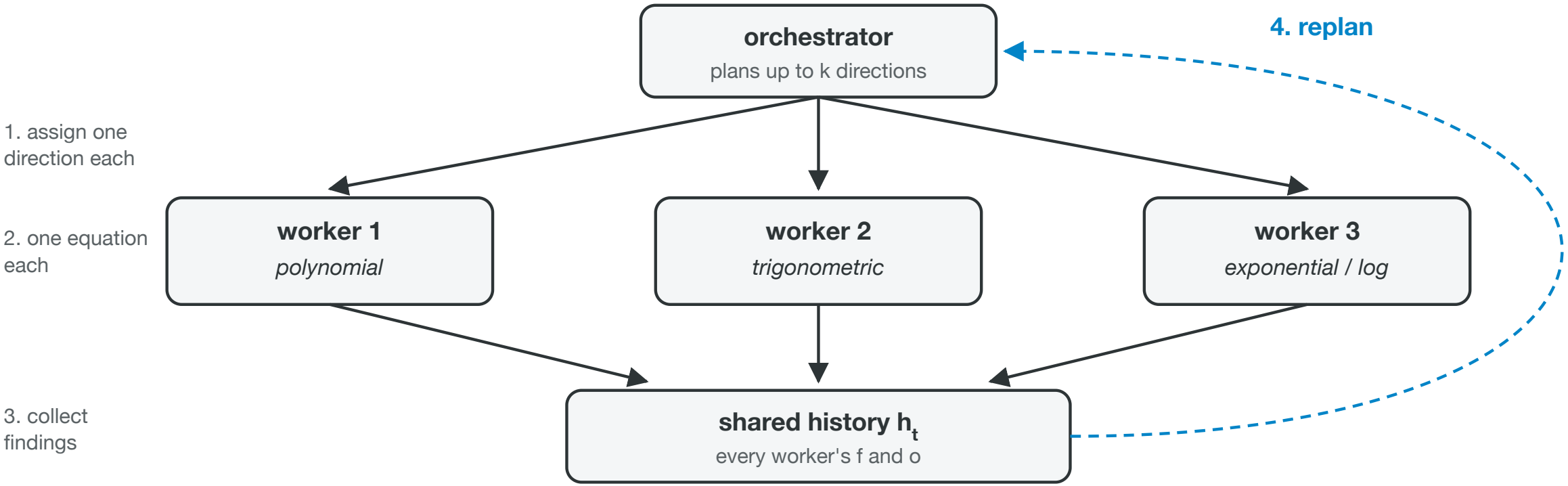
# AutoResearch: what we want to learn

- **Reasoning:** how much does it help over a naive baseline with no reasoning?
- **Long runs (test-time compute):** do more experiments keep improving the result?

What else would we like to understand about this design choice?

# Plan-and-execute: architecture

An orchestrator plans several research directions and assigns one to each worker.



# Plan-and-execute: the algorithm

Each round proposes a set of investigations, runs them, and replans from their results.

$$P_t = \{p_{t,1}, \ldots, p_{t,k}\} \sim M_{\text{orch}}(\cdot \mid q, h_t), \qquad f_{t,j} \sim M(\cdot \mid q, p_{t,j}),$$

$$o_{t,j} = \texttt{evaluate_equation}(f_{t,j}), \qquad h_{t+1} = h_t \cup \{P_t, (f_{t,j}, o_{t,j})_{j=1}^{k}\}$$

```
h = [task]
while budget left and not finished:
    P = M_orch(h)                          # model call: up to k research directions
    F = [M(p) for p in P]                  # one model call per worker: one equation each
    O = [evaluate_equation(f) for f in F]  # fit on training data
    h.append((P, F, O))                    # shared history; replan next round
return best f in h
```

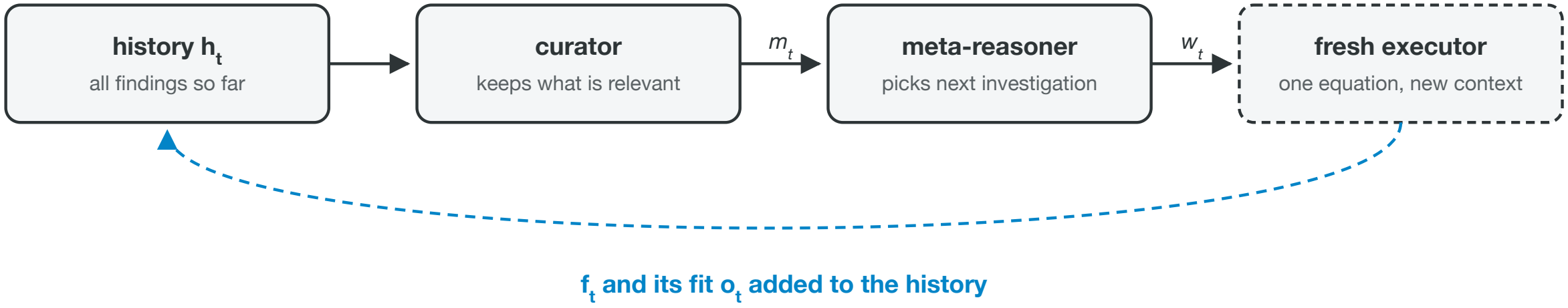
# Plan-and-execute: what we want to learn

- **Speedup:** do $k$ workers converge $k$ times faster, in about $1/k$ of the rounds?
- **Task distribution:** does the orchestrator send different workers to explore different options?

What else would we like to understand about this design choice?

# Context-managed research: architecture

A meta-reasoner picks the next investigation; a fresh executor runs it.



# Context-managed research: the algorithm

Only a curated view of the history reaches each decision.

$$m_t \sim M_{\text{cur}}(\cdot \mid h_t), \quad w_t \sim M_{\text{meta}}(\cdot \mid m_t), \quad f_t \sim M_{\text{exec}}(\cdot \mid q, w_t),$$
$$o_t = \texttt{evaluate_equation}(f_t), \quad h_{t+1} = h_t \cup \{w_t, f_t, o_t\}$$

$m_t$: the context for this decision, built from the history · $w_t$: a work order for the executor

```
h = [task]
while budget left and not finished:
    m = M_cur(h)             # model call: build the context for this decision
    w = M_meta(m)            # model call: choose the next investigation
    f = M_exec(w)            # model call, fresh context: one equation
    o = evaluate_equation(f) # fit on training data
    h.append((w, f, o))      # no model call: the history keeps everything
return best f in h
```

In MIRA, each step runs a full investigation: the executor is an agent with many tool calls. Here it proposes a single equation.

MIRA: Learning What to Investigate Next — Meta-Reasoning for Long-Horizon Research Agents, 2026 (arXiv:2610.02525)

# Context-managed: what we want to learn

- **Curation vs full history:** does curation beat AutoResearch as runs get longer?

What else would we like to understand about this design choice?

# Recap: three architectures

1. **AutoResearch**
2. **Plan-and-Execute**
3. **Context-Managed**

PART 3

# Evaluating harness architectures

# Which metrics should we measure?

- **ID:** $R_{\mathrm{ID}}$ of the final equation $\hat{f}$ on new points in the training range
- **OOD:** $R_{\mathrm{OOD}}$ of $\hat{f}$ on points beyond the training range
- **Tokens:** $T_H$, summed over all model calls

Each is reported against the total budget of each harness.

Which other metrics should we measure?

# In distribution and out of distribution

A good fit on the training range does not guarantee a good fit outside it.

- **ID:** $R^2$ on new points from the training range
- **OOD:** $R^2$ on points beyond it, e.g. train $x \in [-2, 2]$, OOD $x \in [3, 5]$

---

*Report them separately: an equation can interpolate well and still miss the law.*

## Cost

Count every model call, not only the ones that test an equation.

$$T_H = \sum_{\text{calls } c} \left( n_c^{\text{in}} + n_c^{\text{out}} \right), \qquad c \in \{\text{executor}, \text{orchestrator}, \text{curator}, \text{meta-reasoner}\}$$

- Also report: API cost, tool calls, equation evaluations

# Additional metrics: interpretability

An LLM judge scores what a numeric metric cannot.

- Is the equation scientifically meaningful and understandable?
- Is it consistent with the domain description?

We do not cover this metric here.

PART 4 · CASE STUDY

# Three harnesses on LLM-SRBench

# LLM-SRBench and the setup

Same model, tool and data; only the harness changes.

| | |
|---|---|
| **Tasks** | LLM-SRBench LSR-Synth: chemistry, biology, physics, materials science |
| **Model** | `gpt-oss-20b` (open weights) via OpenRouter, for every role |
| **Tool** | `evaluate_equation`: least-squares fit of up to 8 constants |
| **Experiment 1** | AutoResearch with vs without reasoning: 129 tasks, 5 evaluations |
| **Experiment 2** | Plan-and-execute with $k = 1, 2, 4$ workers: 40 tasks, 32 evaluations |
| **Seeds** | 2 per task and harness |

# Automatic harness design

$$\max_{f} R(f) \quad (\text{solve a task}) \qquad \text{vs} \qquad \max_{H} \mathbb{E}_{q}\left[R(H, q)\right] \quad (\text{design a harness})$$

- **AFlow:** searches over workflows written as code, using evaluation feedback
- **ADAS:** a meta-agent writes new agent designs as code and keeps the best ones

Zhang et al., *AFlow: Automating Agentic Workflow Generation*, ICLR 2025
Hu, Lu and Clune, *Automated Design of Agentic Systems*, ICLR 2025

# Summary

Architectural complexity must be justified empirically.

- **Architectures:** AutoResearch, plan-and-execute and context-managed differ in mechanism and cost
- **Metrics:** ID fit, OOD fit and cost can favor different designs
- **Takeaway:** there are many nuanced design questions, and many ways to evaluate them

With frontier models already tuned to their developers' harnesses, does a custom harness beat a well-written instructions file?